

Rahnuma

रहनुमा رہنما

A working guide to the ILM, PANINI and Zistgah stack, from bytes and scripts to compilers, Pāṇini and quantum substrates

प्रकृति

	written	Devanagari hub	Romenagri	and back
Devanagari	प्रकृति	→ प्रकृति	→ prak_ri_ti	→ प्रकृति
Telugu	ప్రకృతి	→ प्रकृति	→ prak_ri_ti	→ ప్రకృతి
Bengali	প্রকৃতি	→ प्रकृति	→ prak_ri_ti	→ the same identity
Urdu, as usually written	ہندوی	→ हनदवी	→ _hana_dawee	→ हनदवी
Urdu, with zer and sukun	ہندویٰ	→ हिन्दवी	→ _hin_dawee	→ हिन्दवी

One identity, many scripts. Telugu and Bengali meet Devanagari in one hub, become the same Romenagri word, and come back exactly. Urdu meets the hub at what is written: without its short-vowel marks the word reads hanadawee; with zer and sukun it is hindawi. Every cell here was produced by the Hindawi tools while this guide was built.

Abhishek Choudhary, AyeAI. Version 2.0.0, 3 October 2026.

Rahnuma, the edition for: Artist, writer or maker

Contents

Part I: Orientation 2

3. Start here: find your way in	3
---------------------------------------	---

Part II: One architecture 5

6. Architecture: core, projections, layers	6
8. The ecosystem, term by term	11

Part III: Setting up 16

15. Running every component	17
-----------------------------------	----

Part V: Working with the estate 29

24. AI, from MYCIN to transformers, and how to use any AI here	30
--	----

Appendices 33

E. For practitioners: teachers, policy makers, communities, makers and engineers	34
--	----

Part I: Orientation

3. Start here: find your way in

In plain words. Four steps to find your part of the guide: who you are, your board or field, what you want to do, and your path with a PDF made for you.

This guide serves a school student and a senior researcher; a village health worker and an investor. Nobody needs all of it. Four steps find your part, on the site or with this page:

1. **Who you are.** A school student, a college student, a postgraduate or doctoral researcher; a teacher; an academic researcher; a working professional, someone in public service, policy or law, someone in community, NGO or village work, an artist, writer or maker; an industry partner or investor; or an AI agent working for a person.
2. **Your board or your field.** A school student chooses a board, ICSE, ISC, CBSE, IGCSE, IB, a state board or another; and a class. Everyone else chooses a field from the international standard classification of fields of study, ISCED-F 2013, the same one FAKIR's lattice uses. It places each subject where its own scholars would: political science is under social and behavioural sciences, not the humanities; law is under business, administration and law; linguistics is under languages.
3. **What you want to do.** Learn the basics, run the software, study or research, teach, contribute, or adopt, fund or invest.
4. **Your path and your edition.** The site lists your chapters in order, with reading times and the components to try first, and offers a PDF edition made for you. Every edition below is a printed selection from this one guide, so all of them stay in step.

Edition	PDF
School student	role-school.pdf
College student	role-college.pdf
Postgraduate or doctoral researcher	role-pg.pdf
Teacher or lecturer	role-teacher.pdf
Academic researcher	role-researcher.pdf
Working professional or engineer	role-professional.pdf
Public service, policy or law	role-public.pdf
Community, NGO or village work	role-community.pdf
Artist, writer or maker	role-creator.pdf
Industry partner, investor or funder	role-industry.pdf
Generic programmes and qualifications	field-00.pdf
Education	field-01.pdf
Arts and humanities	field-02.pdf
Social sciences, journalism and information	field-03.pdf

Edition	PDF
Business, administration and law	field-04.pdf
Natural sciences, mathematics and statistics	field-05.pdf
Information and Communication Technologies (ICTs)	field-06.pdf
Engineering, manufacturing and construction	field-07.pdf
Agriculture, forestry, fisheries and veterinary	field-08.pdf
Health and welfare	field-09.pdf
Services	field-10.pdf

The first public version of this guide, written for advanced researchers and senior professionals, stays as it was at [version 1](#).

Part II: One architecture

6. Architecture: core, projections, layers

In plain words. One shared core, Humanesque, is seen through three cultures: Indian, Persian and Western. Around it sit the parts that turn an idea, in any language, into something real.

One core, three projections

Humanesque is the core: one shared, multidimensional, executable typed hypergraph, with a federated ontology of 26 positions, O_0 to O_{25} , a common execution and evidence spine, and components that can be implemented independently. Its GitHub organisation is hmnsq, where every core stem repository is to live.

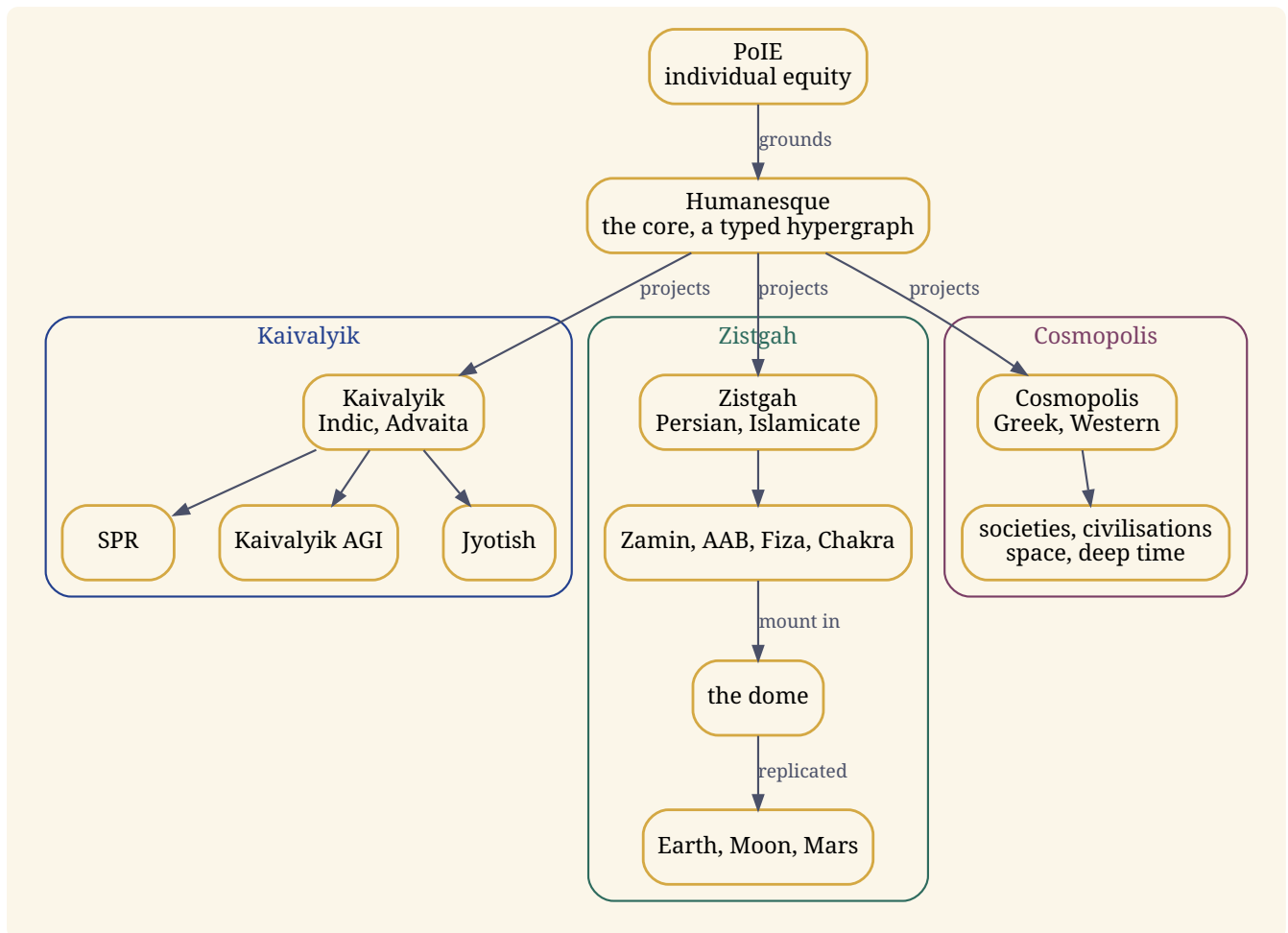


Figure 2. Humanesque and its projections: parallel cultural traversals, not stages.

Kaivalyik, Zistgah and Cosmopolis are three projections over it, running in parallel: the Indic, including the Advaita framing; the Persian and Islamicate; and the Greek, Western and classical. A projection is a domain-specific view and traversal of the whole architecture, with its own culturally sensitive naming and branding. The projections are not stages, not levels of a hierarchy and not versions of one another; they intersect, diverge and reconverge, and none completes another. Zistgah is not the parent of the other two. Their organisations are kaivalyik, zistgah and c-polis; today zistgah carries the working repositories.

The PANINI stack

Layer	What it holds	Where to look
Front end	The languages a program or a prompt can arrive in, Sanskrit first: ILM and the language parsers	project-ilm/ilm.codes, zistgah/tajziya
Middleware	PANINI's own front ends, from <code>ada.pni</code> to <code>zig.pni</code> and <code>sysml.pni</code> , and the PANINI language itself	zistgah/panini, zistgah/humanesque
Core PANINI	The construct model: common semantics plus decorators	zistgah/humanesque, <code>ilm/constructs.csv</code> and <code>ilm/decorators.csv</code>
Realization backends	Where a program meets a substrate: host toolchains for code; PANINIQ for oscillator and quantum substrates; PANINIB for biology; PANINIphy for physical systems	zistgah/paniniq, zistgah/paninib, zistgah/paniniphy

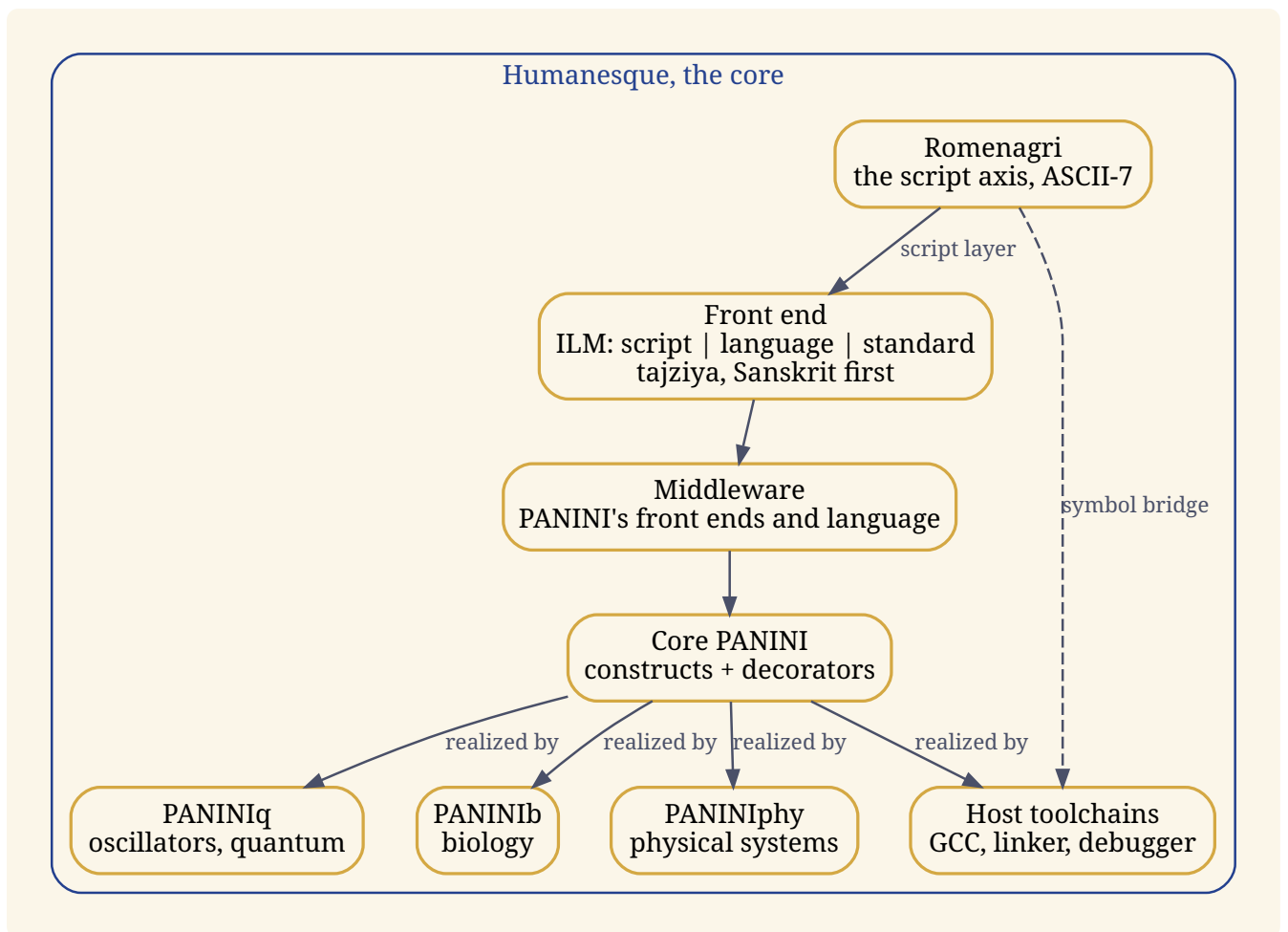


Figure 3. The PANINI stack, with Romenagri as the script layer beneath every front end.

Beneath every front end lies the script layer, Romenagri. Every keyword and every name of a program becomes a word over A to Z, a to z and the underscore, reversibly, so that the whole existing toolchain, from lexer to linker to debugger, carries it unchanged, and the inverse renders it back in the script wherever a person reads it. Chapters 18, 19 and 21 explain it and run it.

A construct such as a counted loop is defined once in the core and translated once per human language. A decorator carries what a particular host language drags in with it, such as `stdio.h` for printing in C. Core PANINI, in his words, “does not violate anything”; the front ends and the realization backends are all projections of it.

How the parts compose

Humanesque	the core; Kaivalyik, Zistgah and Cosmopolis project it in parallel; everything below lives in it, nothing below leads to it
front end	ILM (script language standard) and tajziya, Sanskrit first the script axis is Romenagri, which persists as the symbol bridge through compiler, linker, ELF, DWARF and debugger
middleware	PANINI's own front ends and the PANINI language
core	core PANINI: construct model = common semantics + decorators
realization	host toolchains for code; PANINIq (oscillator, quantum); PANINIb (biology, down to sequence IR); PANINIphy (physical systems); PRATIK and Zamin as a balanced-ternary substrate

Around the stack: the cyclers are written in PANINI and run by one engine; GENIE composes and runs research cycles; AAB paints tasks and runs the process cyclers; FAKIR, UKOP and Dhancha supply the domain lattice and its invariants; Mez, the Cognitive Workbench, composes any of them for an exercise without absorbing them; the provenance systems seal every artifact; VGC gates the work between people and AI. Two readings must be kept apart. As genealogy, the order is Romenagri, Hindawi, ILM, PANINI and the realization arms. As architecture, Romenagri sits inside ILM's script axis, the cyclers are written in PANINI rather than downstream of it, and Humanesque is the core that contains the whole, not its end point.

The realization spine

The realization repositories state the spine they share. PANINI lowers declarative intent through language, semantic IR, a realization requirement, domain resolution and a domain IR, and then a common layer of geometry, trajectory, animation, observation, verification and an ArtifactGraph. PANINIb lowers biological intent through semantic, resolution, CISC, RISC and sequence IRs and emits two coupled artifacts, a physical realization plan and a digital execution run; PANINIphy lowers physical intent through a physical IR and physical-domain engines. “IR contracts are the product. Laboratories and machines are external adapters.” And the same repositories keep the states of a result apart: desired, predicted, observed and validated phenotype are four different things, as are compilation success, execution success, target satisfaction and biological validation.

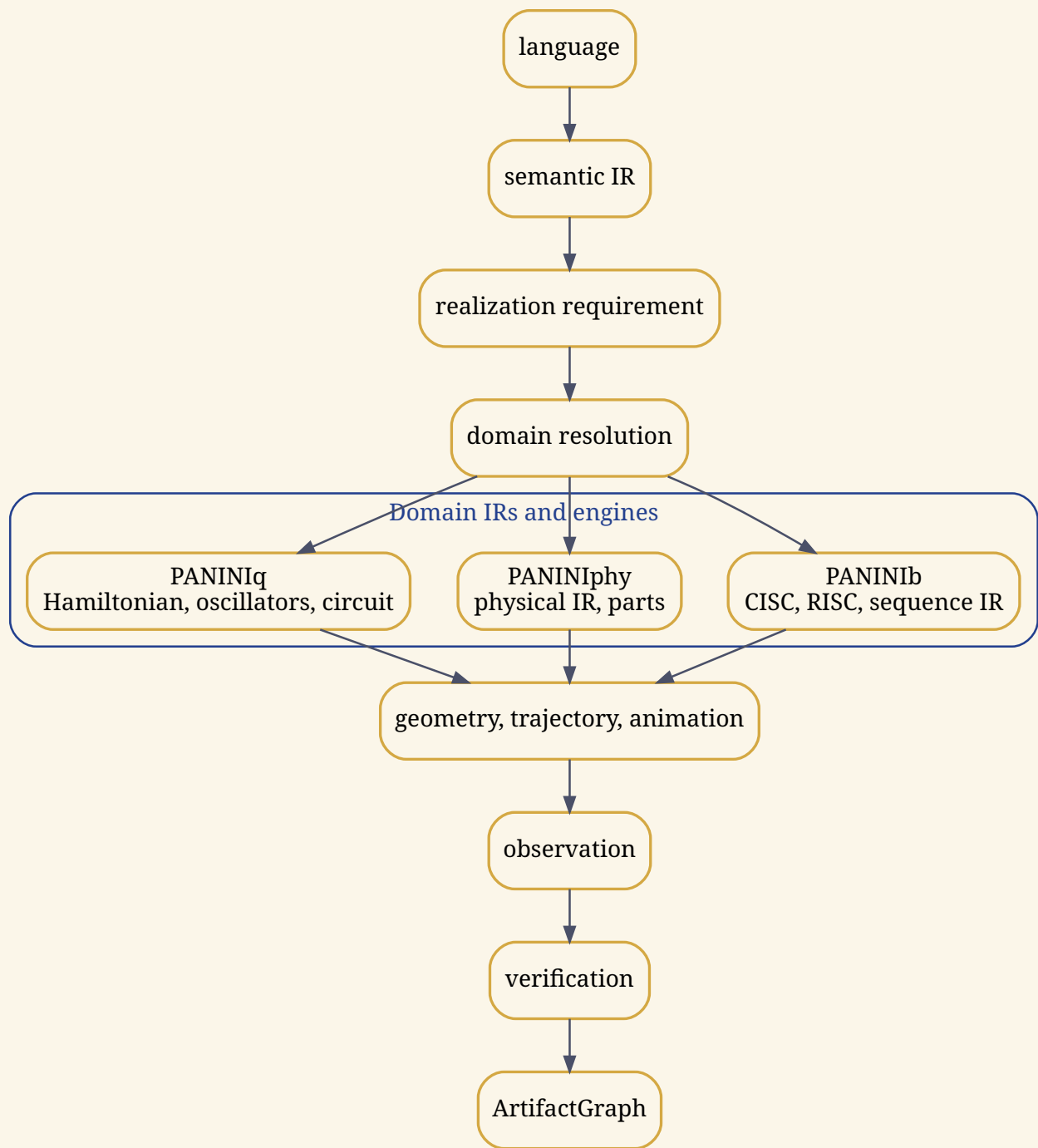


Figure 4. The realization spine the realization arms share.

The habitat elements

Zamin is the ground: the balanced-ternary hardware of PRATIK. AAB is the water: the paint program for systems, from painted intent to verified, sealed artifact, where the verification-gated quests run. Fiza is the air: the environmental replica. Chakra is the turning sky: the observatory and its calendars. All of them mount in one shared virtual dome, which the habitats on the Moon and Mars replicate.

The cyclers

A cycler is a sequence of prompts and the outputs they produce, written down so that it can be edited, shared and run with any AI. Six are classified by what they produce: matba (print), khwab (visual), awaz (audio), tilasm (immersive), pench (embodied) and yadein (the record); genie runs research cycles. Each has its own purpose, contract, context, state model, invariants, failure modes, evidence requirements, workflow and artifact model; only the engine is shared. The protocol behind every cycle, in the author's words: intent, then context, then a meaningful prompt; any AI answers and the human inspects; the artifacts and responses are kept under configuration management; the next prompt follows, until a final artifact that the human authored with intention.

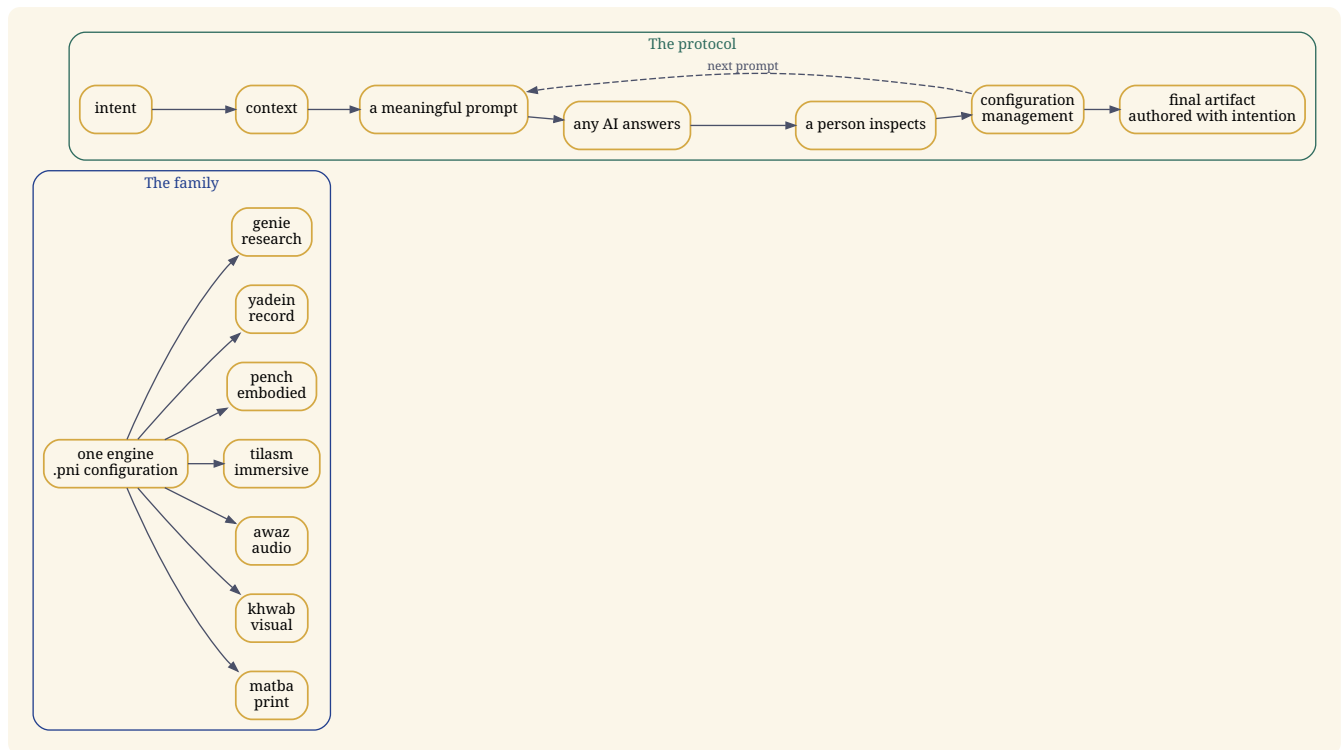


Figure 5. The cyclers: one engine, configured in PANINI, and the protocol every cycle follows.

Provenance, in a fixed order

Four systems, never collapsed into one. **Tok DOI** proves that a thing existed by a given time. **spiguard** is the disclosure gate, which fails closed. **Candor** records signed intent as in-toto statements. **Misty DOI** is the one that reaches the world, through Zenodo. The order is always seal, clear, attest, mint. Chapter 25 explains the machinery.

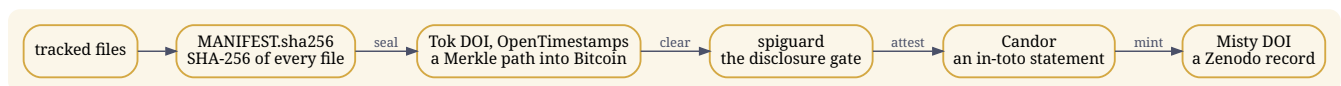


Figure 6. Provenance in its fixed order: seal, clear, attest, mint.

8. The ecosystem, term by term

In plain words. A dictionary of the special words used here, each with the nearest everyday idea and what makes it different.

Each entry gives what the thing is in this architecture, the nearest analogue a newcomer is likely to reach for, and the exact difference. The analogue is a door into the idea, never a definition of it. The state in brackets says what exists: established (historical record), released (on the forge, with its DOI where minted), executed (run for this guide), under construction, specification, or planned. Entries marked † follow the author's working notes and write-ups as supplied on 29 September and 3 October 2026, rather than the repositories.

Language and identity

ILM, Integrative Linguistic Multiscript (released; ilm.codes live). *In the architecture:* the linguistic pillar and the identity layer for language, with script, language and standard kept as separate axes, and layers for phonology, transliteration, orthography, lexicon, syntax and semantics, and interfaces, supported by data, validation, development tools, language specifications and a language server; its registry lists 7,867 languages and 226 scripts. *Nearest analogue:* localisation, an NLP toolkit, a transliteration library. *Difference:* it keeps the three axes apart, so that any language can travel in any script, and it gives every script a reversible path through the whole computing stack, down to the symbol table.

Romenagri (established 2003; released; executed). *In the architecture:* the reversible kernel between scripts and ASCII-7. Its paper states six constraints met together: ASCII-7 closure, case independence, diacritic independence, legality as a C identifier, reversibility and linear time, over the Brahmi, Perso-Arabic and Northwest Semitic families, with a canonical-form layer and an ASCII-reduction layer. *Nearest analogue:* romanisation, such as IAST, ISO 15919 or ITRANS. *Difference:* its output is a valid identifier for every tool from lexer to debugger, and it inverts, so a name survives compilation and comes back.

HPS, the Hindawi Programming System (established 15 August 2004; released; executed). *In the architecture:* nine shailis over nine host languages, from assembly, lex and yacc to C++, Java, Python, BASIC and LOGO; a script layer, a language layer and the host standard, composed as transducers over unmodified host toolchains; now HPS's Indic projection under ILM, with an Urdu edition beside it. *Nearest analogue:* a localised programming language. *Difference:* a complete systems stack, not a keyword translation, with names carried through Romenagri.

tajziya (released, doi:10.5281/zenodo.22982238). *In the architecture:* the parser federation for the classical languages, by family, with the author's Sanskrit parser; the front end of the PANINI stack beside ILM. *Nearest analogue:* NLP parsers. *Difference:* organised to feed PANINI's construct model and AAB's paintings.

Computation

PANINI (several members released; some under construction). *In the architecture*: the construct model at the core, with ILM and tajziya in front, PANINI's own front ends and language as middleware, and realization backends behind; it runs through all three Acts. Its family includes the prompt-cycle language in which the cyclers are written (zistgah/panini), the general-purpose PANINI of the merged release (zistgah/humanesque), and the realization arms. *Nearest analogue*: a compiler, a programming language, an intermediate representation. *Difference*: programming languages, prompt cycles and physical, biological and quantum realizations are all projections of one construct model.

PANINIQ (released, doi:10.5281/zenodo.23020648; executed). *In the architecture*: a realization backend: an oscillator substrate, the PEDLER engine and quantum validation. *Nearest analogue*: a quantum simulator. *Difference*: the substrate is an oscillator Ising machine with a learning automaton on it; the quantum circuit validates it rather than replacing it.

PANINIB and **PANINIPHY** (released research platforms, version 1.0). *In the architecture*: PANINIB, the biological realization arm, lowers biological intent through semantic, resolution, CISC, RISC and sequence IRs to a realization plan and an execution run; PANINIPHY, the physical realization arm, lowers physical intent through a physical IR and physical-domain engines, down to modular blocks and the selection of parts by force, stroke and voltage. *Nearest analogue*: computer-aided design, genome design tools. *Difference*: both share PANINI's common spine, language, verification, provenance and ArtifactGraph, adding only their own domain IR and engines; laboratories and machines are external adapters, and neither claims a result its evidence does not show.

Cognition

PEDLER and **QEDLER** (established November 2001; record doi:10.5281/zenodo.17497559; QEDLER a research framework). *In the architecture*: the point-event-driven learner; a six-tuple extension of the Turing machine over balanced ternary; the source of the primitives Act (executable intent), Inclination (a signed directional gradient on transition edges) and Natural Justice (a gate on the passage from intent to Act); QEDLER extends it to physics. *Nearest analogue*: an event-driven or neuromorphic learning model. *Difference*: it predates the present generation of models and carries the executable-intent semantics the constitutional corpus is built from.

PRATIK and **Zamin** (released: pratik_core_mvp doi:10.5281/zenodo.21288232, zamin doi:10.5281/zenodo.21297556). *In the architecture*: PRATIK, Participatory Recursive Adaptive Trans-Intelligence Kernels, on the divider between Acts I and II, with a kernel in C++ and CUDA; Zamin, its physical balanced-ternary substrate with a poised zero. *Nearest analogue*: a neuromorphic kernel on novel hardware. *Difference*: balanced ternary and event-driven from the logic up.

CEM, with CEMb, CEMs and Eco-CEM (specification; Act II). *In the architecture*: cognitive enablement modules over one kernel, $\mathcal{E} = (\mathbf{E}, \mathbf{C}, \mathbf{\Pi}, \mathbf{W})$, realised on biological substrates (BCI, prosthetics, closed-loop cognition, operated through Interglial Healthcare), on synthetic ones (processors, distributed systems, robotics) and on ecological ones. *Nearest analogue*: robotics, BCI, prosthetics. *Difference*: one invariant kernel across substrates; the superscript is part of the name.

The AyeAI Triad (specification with formal closure). *In the architecture*: AyeAM = $\langle \mathbf{S}, \mathbf{R}, \mathbf{C} \rangle$ for embodiment, AyeAI = $\langle \mathbf{M}, \mathbf{I}, \mathbf{G} \rangle$ for cognition, AyeCNSe = $\langle \mathbf{T}, \mathbf{Ch}, \mathbf{\Sigma} \rangle$ for coordination and communication across media, with AyeAI at the apex. *Nearest analogue*: a perception, cognition and

action loop. *Difference*: a triad, not a loop and not a pipeline.

Knowledge and agency

UKOP, FAKIR and Dhancha (FAKIR released, doi:10.5281/zenodo.21436550; Dhancha released, doi:10.5281/zenodo.22821645; UKOP a specification). *In the architecture*: UKOP, the reference architecture for human knowledge; FAKIR, its kernel over ISIC, ISCO and ISCED crossed with AGI layers and language; Dhancha, the spine every domain is built on, whose rule is that the engine is common and the workflow is not. *Nearest analogue*: a knowledge graph or reasoning engine. *Difference*: domain invariants are enforced by tests, and resolution and verification are primitives.

Cyclers (six released and minted; Act I). *In the architecture*: matba, khwab, awaz, tilasm, pench and yadein, classified by what they produce, plus genie, each with its own contract, state model and evidence rules, written in PANINI and run by one engine. *Nearest analogue*: an agentic harness or orchestration loop. *Difference*: AI-agnostic by construction, inspected by a person at every step, and the recorded method, never the content, is itself the reproducible work. † An older form of the corpus names SAFAR in the sixth position, where the current corpus has yadein.

Kitab (released). *In the architecture*: the config-driven book template that matba publishes through, and † in the author's notes the persistent artifact and publication layer that keeps an artifact's state, its creative lineage, its execution trace and evaluations, its forks and descendants. *Nearest analogue*: a publishing template. *Difference*: a cycler publishes through Kitab; Kitab is not a seventh cycler.

Research Kundali (released). *In the architecture*: a research map of a person, a lab or a paper, built from an ORCID, a lab name or a DOI ([project-ilm/research-kundali](https://project-ilm.com/research-kundali)); † in the author's notes, the systematic record of claims, artifacts, chronology, provenance, objections, evidence and responses, so that every claim can be checked against its evidence. *Nearest analogue*: a researcher profile. *Difference*: it records evidence and discrepancies, not reputation.

GENIE (released). *In the architecture*: the Generalized Emotive-Narrative Interaction Engine, storyteller, poet and painter, and the research cycle through its Prompt Operating System, with six verbs (create, verify, execute, measure, falsify, integrate), nine epistemic tags and a gate that a simulation cannot pass in place of an experiment. † In the author's own formulation GENIE is itself a cycler, the creative cycler, which can decorate the modality cyclers without erasing their identities. *Nearest analogue*: a creative or research AI agent. *Difference*: governed by a constitution of primitives; orchestration and composition, not an implementation.

AAB (released; Act I). *In the architecture*: the paint program for systems, from painted intent to verified, sealed artifact; the gamified studio of verification-gated quests; home of the process cyclers; the process half of the estate's software factory, with FAKIR as its component registry. In the author's words, AAB and FAKIR together constitute the working definition of AGI used here. *Nearest analogue*: a low-code studio or a software factory. *Difference*: every task is a painting with a manifest, quest stages and oracle-gated verification, and every component keeps its provenance.

Mez, the Cognitive Workbench (released). *In the architecture*: the local-first desk that brings independently existing systems together for an exercise: composition, exercise, observation, synthesis, a new artifact. *Nearest analogue*: an IDE or a notebook. *Difference*: composition without absorption; every composed system keeps its own identity and life.

Method and evidence

VGC, Verification-Gated Human-AI Co-Development (released, doi:10.5281/zenodo.21264248). *In the architecture*: the method by which people and AI build together, with every step gated by verification. *Nearest analogue*: test-driven development or human review of AI output. *Difference*: the gate is a verified artifact, not a reviewer's impression, and the AI's report that something passed is never the evidence that it passed.

COPA, the Cost of Perceived Authority (released, doi:10.5281/zenodo.21782217). *In the architecture*: a protocol for measuring how much an AI system asserts before it verifies, through the Authority Projection Index, with its hypothesis stated beside a fair null. *Nearest analogue*: an AI evaluation benchmark. *Difference*: it measures projected authority rather than accuracy, and its target is calibrated trust.

† One rule binds all of these: a record that has not been retrieved is not thereby false. Unretrieved is a state of the reader's context, not of the world, and a claim moves from unknown to false only on evidence.

Continuity and civilisation

Humanesque (specification; the merged release on the forge). *In the architecture*: the core over which everything above is a typed hypergraph, and the constitutional realm derived from PoIE through recursive sovereignty. *Nearest analogue*: an AGI architecture. *Difference*: a constitutional and epistemic realm as well as a technical substrate, not a product.

Kaivalyik, Zistgah and Cosmopolis (projections). As in Chapter 6: parallel cultural projections of the whole, not stages. Kaivalyik carries Synthematic Pragmatic Realism, SPR, the framework that heads Act III ("Kaivalyik: Towards a Synthematic Pragmatic Realism for the AGI Singularity"); Zistgah carries the habitats and the elements.

TransEg (released, doi:10.5281/zenodo.21321558). *In the architecture*: the transferred alter ego, the mechanism by which a constituted identity continues; a local digital-twin reference implementation; yadein's staggered upload feeds it. *Nearest analogue*: a digital avatar. *Difference*: continuity of an identity under the constitutional corpus, not a persona.

PAT.AL, VIDYA, GramSheel, Project VIKRAM and TWISHA (established). PAT.AL, the Participatory Alliance for Technology, Access and Livelihoods, is participatory infrastructure; VIDYA bridges to AyeAM; Project VIKRAM, Virtualized Infrastructure for Knowledge-driven Rural Ascension Management, runs under the GramSheel Foundation beside TWISHA. They sit on the dividers between the Acts. The question the author puts to them is whether technology can distribute capability without concentrating power.

The reference lab (planned; its launch scripts are the next release). An AGI-capable laboratory for 30 to 40 thousand, able to carry 80 to 90 per cent of the research behind high-end AI, robotics and automation papers, repeated as a pattern for every domain and aligned with GATE-level courses in computer science and in robotics and automation.

Also in the working notes

Modular self-reconfiguring robots †: very simple modular blocks with face-to-face connections and electro-permanent-magnet contacts, assembling into larger structures, ruggedised for decentralised manufacture with low-cost embedded control; PANINIphy's modular-block examples are the compiler side of this. **Further realization families** †: chemical, photonic, and a computational-biological intermediate layer of the AlphaFold class, built as an independent spine rather than a dependency. **The build engine** †: the direction of the whole is a self-reflecting, self-evolving and self-sustaining build engine in which PANINI captures and expresses the construct and the intent, and agency, human or machine, biological or physical, does the building. **INDUS and KHĀK** † are named in the notes among the lineages and the robots, alongside ANGEL, TARA and NI2A2.

Part III: Setting up

15. Running every component

In plain words. Step-by-step commands to start each part on your own computer, with what you should see when it works and what to do when it does not.

Every component below was cloned and run on a clean machine on 3 October 2026, with the commands its own README gives, and each card says exactly what happened. Where a step needed something the README does not mention, the card adds it; where a component did not pass, the card says why. Set up the machine first (Chapters 13 and 14).

Each card has the same shape: what the component is, what it needs, the commands to type in order, what you should see, and its status. Every card starts with a clone of the repository into a folder of the same name; the commands that follow are typed inside that folder.

Machine: Ubuntu 24.04, Python 3.12, Node 22, GCC 13. *Run on:* 3 October 2026.

Language and identity

Hindawi and Romenagri

What it is. The Hindawi Programming System's C front end, its driver hincc and the Romenagri script layer. *Needs:* gcc, make, flex, libfl-dev, gawk.

```
git clone https://github.com/hindawiai/chintamani.git && cd chintamani
mkdir -p .root/usr/bin .root/usr/lib .root/usr/include
make -C Romenagri all && make -C Romenagri install INSTRROOT="$PWD/.root"
export PATH="$PWD/.root/usr/bin:$PATH"
make -C Hindawi/guru all && make -C Hindawi/guru install INSTRROOT="$PWD/.root"
make -C Hindawi/hindr install INSTRROOT="$PWD/.root"
cp Hindawi/samples/HindiC.uhin . && hincc HindiC.uhin
printf 'रम\nरम\n' | ./hin.exe
```

You should see: संकलन के परिणाम, then the program asks आपका नाम क्या है? and counts to ten. *Status:* Executed for this guide. The compilers chapter prints the run.

The Urdu edition

What it is. Urdu shailis and the driver urducc, which sends an Urdu-script program through the Devanagari hub and Romenagri. *Needs:* the Hindawi build above, on the PATH.

```
git clone https://github.com/zistgah/urdu-ilm.git && cd urdu-ilm
mkdir -p .root/usr/bin .root/usr/lib .root/usr/include
make -C Romenagri all && make -C Romenagri install INSTRROOT="$PWD/.root"
export PATH="$PWD/.root/usr/bin:$PATH:<chintamani clone>/.root/usr/bin"
make -C ILM/guru all
bash ILM/urducc -s ILM/samples/UrduC.uhin
bash ILM/urducc ILM/samples/UrduC.uhin && ./hin.exe
```

You should see: C with the names na and ka, then the prompt in the hub script. *Status:* Executed for this guide. Replace with the folder of the Hindawi clone; the compilers chapter prints the run.

tajziya

What it is. The parser federation for the classical languages, by family. *Needs:* python3, make.

```
git clone https://github.com/zistgah/tajziya.git && cd tajziya
PYTHONPATH=src python3 -m tajziya commonalities
make check
```

You should see: make check: 19 passed, 0 failed, 1 unjudged. *Status:* Passed when run on 3 October 2026. Run the module with PYTHONPATH=src, from the clone's root.

The PANINI family

PANINI, the prompt-cycle language

What it is. The language the cyclers are written in, with its engine and studio. *Needs:* python3, node.

```
git clone https://github.com/zistgah/panini.git && cd panini
python3 panini.py check cyclers/pench.pni
python3 panini.py stages cyclers/yadein.pni
node tests/panini.test.mjs
python3 panini.py serve
```

You should see: 0 problems; 45 pass, 0 fail; the studio answers on 127.0.0.1:8717. *Status:* Passed when run on 3 October 2026.

PANINI, parser and interpreter

What it is. A general-purpose PANINI implementation with its conformance report. *Needs:* node.

```
git clone https://github.com/zistgah/panini_by_claude.git && cd panini_by_claude
node bin/panini.mjs conformance
npm test
```

You should see: 145 passed, 0 failed. *Status:* Passed when run on 3 October 2026.

PANINI, the stage-0 bootstrap

What it is. PANINI with a bootstrap that records its own evidence. *Needs:* node.

```
git clone https://github.com/zistgah/panini_by_grok.git && cd panini_by_grok
node src/cli.js run examples/hello.pni
node tests/run.mjs
node scripts/bootstrap.mjs
```

You should see: Hello, PANINI; 32 passed, 0 failed; stage-0 evidence written. *Status:* Passed when run on 3 October 2026. Its wider report, npm test, passes 81 of 83 here; the two others need tools this machine lacks.

The Humanesque merged release

What it is. PANINI, ILM, Hindawi, the cyclers and Mez in one tree. *Needs:* bash, node; optionally zig.

```
git clone https://github.com/zistgah/humanesque.git && cd humanesque
bash VERIFY.sh
npm test
```

You should see: 79 of 83 checks pass. *Status:* Partly passes, as run on 3 October 2026. The four that do not pass here need two downloadable model files (wllama.wasm and stories15M.Q4_0.gguf), the zig toolchain, and the Bengali flattening round trip.

PANINIq

What it is. The oscillator substrate, the PEDLER engine and quantum validation. *Needs:* python3, numpy, scipy, qiskit.

```
git clone https://github.com/zistgah/paniniq.git && cd paniniq
python3 -m venv ../venv-paniniq && . ../venv-paniniq/bin/activate
export PYTHONDONTWRITEBYTECODE=1
pip install -r requirements.txt qiskit
python3 -m unittest discover -s tests
python3 demo.py
bash ops/verify.sh
```

You should see: Ran 23 tests, OK; its gate 10 passed. *Status:* Executed for this guide. The first-run chapter prints the run.

PANINIB, biological realization

What it is. Biological intent lowered through typed IRs to a realization plan. *Needs:* python3.

```
git clone https://github.com/zistgah/paninib.git && cd paninib
python3 panini compile examples/replace.panini -o out
python3 panini inspect out --stage sequence-ir
python3 panini verify out
```

You should see: each step prints its JSON record. *Status:* Passed when run on 3 October 2026.

PANINIphy, physical realization

What it is. Physical intent compiled through a physical IR, with parts chosen by requirement. *Needs:* python3.

```
git clone https://github.com/zistgah/paniniphy.git && cd paniniphy
python3 panini compile examples/modular_block.panini -o out
python3 panini parts --category actuator --force 50N --stroke 20mm --voltage 12V
```

You should see: physical resolution=RESOLVED verify=True. *Status:* Passed when run on 3 October 2026.

Cyclers, desk and studio

matba, the press

What it is. Posters in, book out, sealed, pushed, minted. *Needs:* python3.

```
git clone https://github.com/zistgah/matba.git && cd matba
python3 matba.py serve
```

You should see: the press answers on 127.0.0.1:8710. *Status:* Passed when run on 3 October 2026.

khwab, the cutting room

What it is. The visual cycler. *Needs:* python3.

```
git clone https://github.com/zistgah/khwab.git && cd khwab
python3 khwab.py serve
```

You should see: the cutting room answers on 127.0.0.1:8711. *Status:* Passed when run on 3 October 2026.

awaz, the listening room

What it is. The audio cycler. *Needs:* python3.

```
git clone https://github.com/zistgah/awaz.git && cd awaz
python3 awaz.py serve
```

You should see: the listening room answers on 127.0.0.1:8712. *Status:* Passed when run on 3 October 2026.

tilasm, pench and yadein

What it is. The immersive, embodied and record cyclers; each repository has the same tests and its own page. *Needs:* node.

```
git clone https://github.com/zistgah/tilasm.git && cd tilasm
node tests/cycler.test.mjs
node tests/inbox.test.mjs
node tests/models.test.mjs
```

You should see: 20, 23 and 22 pass, 0 fail. *Status:* Partly passes, as run on 3 October 2026. Run the same three in zistgah/pench and zistgah/yadein. Two further test files, genie.test.mjs and leak.test.mjs, import modules the repositories do not carry.

GENIE

What it is. The Generalized Emotive-Narrative Interaction Engine. *Needs:* node.

```
git clone https://github.com/zistgah/genie.git && cd genie
node test/run.js
```

You should see: 191 passed, 0 failed. *Status:* Passed when run on 3 October 2026. Open index.html, or its page, for the engine itself.

alam

What it is. Small work, cycled with any AI. *Needs:* node, npm.

```
git clone https://github.com/zistgah/alam.git && cd alam
npm install jsdom
node test/alam-test.js
node test/dom-test.js
```

You should see: alam dom: 9 passed, 0 failed. *Status:* Passed when run on 3 October 2026. The page test needs jsdom, installed by the first line.

Mez, the Cognitive Workbench

What it is. The local-first desk. *Needs:* python3.

```
git clone https://github.com/zistgah/mez.git && cd mez
./mez doctor
./mez bearings
./mez serve
```

You should see: no account, no API key, no network required; the desk answers on 127.0.0.1:7373. *Status:* Passed when run on 3 October 2026.

Knowledge, domains and instruments

FAKIR

What it is. The knowledge kernel and its explorer. *Needs:* bash.

```
git clone https://github.com/zistgah/fakir.git && cd fakir
bash ops/verify.sh
```

You should see: CONTRACT SATISFIED, 6 checks. *Status:* Passed when run on 3 October 2026. The explorer is its page: zistgah.org/fakir

Dhancha

What it is. The domain spine every FAKIR domain is built on. *Needs:* bash, python3, make.

```
git clone https://github.com/zistgah/dhancha.git && cd dhancha
make check
```

You should see: site matches the registries; pr_gate bites on a claim and passes a clean change. *Status:* Passed when run on 3 October 2026.

ertabat

What it is. The communication-engineering domain. *Needs:* bash.

```
git clone https://github.com/zistgah/ertabat.git && cd ertabat
bash ops/verify.sh
```

You should see: CONTRACT OK. *Status:* Passed when run on 3 October 2026.

jugaad28

What it is. The 28 nm tape-out catalogue. *Needs:* bash.

```
git clone https://github.com/zistgah/jugaad28.git && cd jugaad28
bash ops/verify.sh
```

You should see: VERIFY GREEN. *Status:* Passed when run on 3 October 2026.

TransEg

What it is. The local digital-twin reference implementation. *Needs:* bash, python3; Docker for the stand-up.

```
git clone https://github.com/zistgah/transeg.git && cd transeg
./scripts/test.sh
./scripts/bootstrap.sh --dry-run
```

You should see: 10 passed; the stand-up plan. *Status:* Passed when run on 3 October 2026. The full stand-up, ./scripts/bootstrap.sh and docker compose, needs Docker and a GPU; it was not run here.

TransEg identity governance

What it is. Typed layers, envelope, staged policies and audit. *Needs:* python3, jsonschema, pytest.

```
git clone https://github.com/zistgah/transeg-idgov.git && cd transeg-idgov
pip install jsonschema pytest
python3 tools/idgov_validate.py policies
python3 -m pytest -q tests/
```

You should see: VALID: 8 stage policies; 14 passed. *Status:* Passed when run on 3 October 2026.

CHAKRA

What it is. The offline observatory and its calendars. *Needs:* node.

```
git clone https://github.com/project-ilm/chakra.git && cd chakra
node test/run.js
```

You should see: ALL SUITES PASSED. *Status:* Passed when run on 3 October 2026. Open index.html for the observatory.

Jyotish

What it is. The panchang, the Kaivalyik projection of CHAKRA. *Needs:* bash, node.

```
git clone https://github.com/zistgah/jyotish.git && cd jyotish
bash ops/verify.sh
```

You should see: PASSED, with 1 check skipped and named. *Status:* Passed when run on 3 October 2026. Open index.html for the panchang.

Cognition and hardware

PRATIK kernel

What it is. The balanced-ternary, event-driven kernel. *Needs:* g++; nvcc for CUDA.

```
git clone https://github.com/zistgah/pratik_core_mvp.git && cd pratik_core_mvp
bash build.sh
bash build.sh cuda
```

You should see: ALL CHECKS PASSED. *Status:* Passed when run on 3 October 2026. The CPU build passed here; the CUDA build needs nvcc and an NVIDIA GPU. The script is not marked executable, so call it with bash.

Zamin

What it is. Ternary silicon: SPICE cells, layout and a simulator. *Needs:* ngspice, python3.

```
git clone https://github.com/zistgah/zamin.git && cd zamin
cd spice && ngspice -b sign_frustration_tb.sp
```

You should see: a SPICE run of the ternary cells. *Status:* Does not run as is, as run on 3 October 2026. On ngspice 42 the testbench stops: the source v_gnd is reported as a shorted voltage source. The interactive simulator is its page.

QEDLER

What it is. The event-hypergraph physics programme. *Needs:* bash.

```
git clone https://github.com/project-ilm/qedler.git && cd qedler
bash ops/verify.sh
bash ops/resume.sh && cat RESUME.md
```

You should see: 5 of 6 clauses pass. *Status:* Partly passes, as run on 3 October 2026. Clause C2, manifest completeness, fails: 15 files are not in MANIFEST.sha256. Resealing the manifest clears it.

Provenance, tools and labs

Misty DOI

What it is. DOI minting and packaging for Zenodo. *Needs:* python3, pip.

```
git clone https://github.com/project-ilm/misty-doi.git && cd misty-doi
pip install misty-doi
misty --help
misty init
misty validate -m misty.json
```

You should see: `usage: misty [-h] [--version] {init,validate,transform,package,ots,publish}`. *Status:* Passed when run on 3 October 2026. Publishing needs a Zenodo token in ZENODO_TOKEN.

Research Kundali

What it is. A research map of a person, a lab or a paper. *Needs:* python3 and network access to the public records it reads.

```
git clone https://github.com/project-ilm/research-kundali.git && cd research-kundali
python3 kundali/kundali.py <ORCID, lab name or DOI> --out out
```

You should see: a research map in out/. *Status:* Not run here: needs network access. It reads public records such as ORCID; it could not reach them from this build machine, and stops rather than write an empty chart.

The linguistics lab

What it is. A starter lab in a container. *Needs:* Docker.

```
git clone https://github.com/project-ilm/linguistics-labs.git && cd linguistics-labs
docker build -t ilm-lab .
docker run --rm -p 8888:8888 -v "$PWD":/lab ilm-lab
```

You should see: the lab on localhost:8888. *Status:* Not run here: needs Docker. Not run here: this build machine has no Docker.

This guide

What it is. The site, the PDF and the Markdown, with their gate. *Needs:* python3, gcc, make, flex, libfl-dev, gawk, gdb; for a rebuild also pandoc, node, graphviz.

```
git clone https://github.com/zistgah/rahnuma.git && cd rahnuma
bash ops/verify.sh
bash examples/hindawi.sh words
python3 tools/build.py
```

You should see: 12 passed, 0 failed, 0 unjudged. *Status:* Passed when run on 3 October 2026.

Components you open in a browser

Component	Page
FAKIR's explorer	zistgah.org/fakir
The shared dome	zistgah.github.io/dome
AAB, the factory process	zistgah.org/aab
GENIE	zistgah.github.io/genie

tilasm, pench and yadein | zistgah.github.io/tilasm |
Kitab | zistgah.github.io/kitab |
Tok DOI | project-ilm.github.io/tok-doi |
CHAKRA | project-ilm.github.io/chakra |
Jyotish | zistgah.github.io/jyotish |
ILM | ilm.codes |
PANINI's studio | zistgah.github.io/panini |
The estate | zistgah.org |

Part V: Working with the estate

24. AI, from MYCIN to transformers, and how to use any AI here

In plain words. How AI grew from rule books to today's language models, and how to use any AI, safely, to help with this work.

Two traditions

The first tradition in AI wrote knowledge down as rules. MYCIN, built at Stanford in the 1970s (Shortliffe, 1976), identified the bacteria behind severe infections and recommended antibiotics with about six hundred production rules of the form *if these findings, then this conclusion, with this certainty*. Each rule carried a certainty factor between -1 and $+1$, and two positive factors for the same conclusion combined as

$$CF = CF_1 + CF_2(1 - CF_1).$$

In evaluations its recommendations stood comparison with those of infectious-disease specialists, yet it was never used on patients, for reasons of integration, accountability and trust rather than accuracy: the first lesson of applied AI, and still the current one. Its name comes from the suffix of the antibiotics it prescribed, streptomycin, erythromycin and the rest, which in turn comes from the Greek *mykēs*, fungus, because the soil organisms that yield many of them were once taken for fungi. Take the medical rules out and the inference engine that remains, EMYCIN, became the pattern for a generation of expert systems.

The second tradition learns from data: perceptrons, then multilayer networks trained by backpropagation, then deep learning. The estate uses both. PANINI and ILM are symbolic, with grammars, constructs and transducers; PEDLER is adaptive and event-driven.

Transformers

A language model reads tokens, not letters. Text is cut into subword units by byte-pair encoding or a similar scheme, learned from a training corpus. Tokenisers trained mostly on English cut Indic text into many more tokens for the same content, and since every Devanagari letter is already three bytes in UTF-8, the same sentence costs more to process, fills the context window sooner and is modelled less well. Script-aware normalisation and tokenisation, of the kind ILM provides, address exactly this.

Each layer of a transformer mixes information between positions by attention,

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V,$$

in several heads at once, with the queries Q , keys K and values V computed from the tokens (Vaswani and colleagues, 2017). The model is trained to predict the next token, maximising $\sum_t \log p_\theta(x_t | x_{<t})$ over a very large corpus, and then tuned on human preferences.

Does AI only pull and pool data?

Partly. A model is trained on existing text, and it can state errors fluently, so everything it says is a claim to be checked, not a fact. But it does not look documents up and paste them: it computes a probability distribution over continuations, and it can apply a method to material it has never seen. The stance that follows is the estate's verification-gated co-development: nothing is accepted without an oracle, whether a test suite, a second implementation, a measurement or a review. The examples in this guide were written with an AI assistant and accepted only because they ran, and their outputs were checked against the sources.

Choosing an AI

Any of them will do for the method below. Chat assistants include Claude, ChatGPT, Gemini, Grok, DeepSeek and Mistral's Le Chat. They differ in their models, context lengths and tools, and they change every few months.

Local models keep everything on your own machine. With Ollama:

```
curl -fsSL https://ollama.com/install.sh | sh
ollama run llama3.2
```

A GPU with 4 GB of memory runs models of a few billion parameters quantised to four bits; llama.cpp and LM Studio are alternatives.

Coding agents work in the terminal, inside a repository, reading files and running commands with your permission. The install commands are each vendor's documented ones at the time of writing:

Agent	Install	Start it inside a repository
Claude Code	<code>npm install -g @anthropic-ai/claude-code</code>	<code>claude</code>
Codex CLI	<code>npm install -g @openai/codex</code>	<code>codex</code>
Gemini CLI	<code>npm install -g @google/gemini-cli</code>	<code>gemini</code>
Aider, with many providers and local models	<code>python3 -m pip install aider-install && aider-install</code>	<code>aider</code>

The method: a cycle with any AI

The estate's cyler protocol works with every one of them.

1. **Intent.** One sentence saying what you want.
2. **Context.** Give the AI the repository's own files before anything else: `README.md`, `CONTEXT.md`, `CONTRACT.md`, `AGENTS.md` where present, and `ops/verify.sh`. Many estate repositories carry these files precisely so that any person or any AI can start cold. For this guide, the whole text is one file, `docs/rahnuma.md`, and `docs/llms.txt` lists its chapters.

3. **A meaningful prompt.** Name the files, the exact change, and the check that will decide whether it is accepted.
4. **Inspect.** Run the tests or the gate yourself and read the diff. The AI's report that something passed is not evidence that it passed.
5. **Keep the record.** Commit the prompt, the response and the result together.
6. **Next prompt,** until the artifact is one you authored with intention.

A prompt that works, for PANINIq:

Here are README.md, CONTRACT.md, tests/test_core.py and ops/verify.sh from zistgah/paniniq. Add one test to tests/test_core.py that checks the order parameter R lies in $[0, 1]$ for 200 random phase vectors. Show the new test, then the output of `python3 -m unittest discover -s tests` and of `bash ops/verify.sh`.

Four rules keep this safe. Never paste a password, token or private key into any AI. Do not let an agent push, publish or mint; those steps stay with a person. Do not accept code you have not run. And for anything current, such as versions, dates or prices, make the AI search, or check it yourself.

Appendices

E. For practitioners: teachers, policy makers, communities, makers and engineers

In plain words. A guide for teachers, policy makers, community workers, artists and engineers: where to start and how to take part.

Teachers and lab builders. Start from the reference lab, the GATE chapter and the component cards, which double as laboratory sheets: each says what to install, what to type and what a student should see (Chapters 16, 26 and 15). The reader views let each student follow a path suited to their age and background, and remember what they have read.

Policy makers and administrators. The questions that matter are in Chapter 10 and Appendix C: sovereignty in its testable technical sense, the ability to rebuild, verify, modify and run (Chapter 21); provenance that makes claims checkable; and infrastructure that leaves capability with communities. The executable contracts of the repositories show what verifiable governance of software looks like in practice.

Community workers and health practitioners. Project VIKRAM's sections cover education, health, livelihood, justice, environment and habitat, food and agriculture, and peace; its health section accepts submissions as issues. The Urdu edition and the Brahmi hub show work in people's own scripts; CHAKRA and Jyotish run offline.

Artists, writers and makers. The cyclers produce print, film, sound, immersive work, embodied work and the record; matba turns posters into a sealed book; GENIE writes and paints; PANINIphy turns a physical idea into parts. Every cycle is recorded as method, so a piece of work can be repeated, shared and credited.

Engineers in industry. Adopt one component at a time: read its card, clone it, run it, read its `CONTRACT.md` and its gate, check its DOI and provenance, then integrate it through its interface. † The notes give the full adoption ladder: the canonical artifact, a one-page explanation, the nearest industry analogue and the exact difference, a working demonstration, the repository, the DOI and timestamp, the interface, a case study, and the commercial route.

Contributors. Fork, open an issue, send a pull request. Every repository's gate runs on your change; every claim must point to its evidence; the AI you use is welcome, and its report that something passed is not the evidence that it did.

© 1993–2026 Abhishek Choudhary. All rights reserved. AyeAI. Text CC BY-SA 4.0; scripts GPL-3.0-or-later. Version 2.0.0, 3 October 2026.